

Assignment Objective:

In this assignment, we will explore the Multi-Layer Perceptron (MLP) architecture and its application to natural language processing.

The main goals are:

1. Build and understand the structure of an MLP
2. Train the MLP on the IMDB dataset for sentiment analysis
3. Experiment with various tokenization techniques
4. Optimize hyperparameters to improve model performance
5. Compare the best-performing model against a random MLP baseline
6. Implement an MLP with feedback alignment

1. Experiments and Results:

1.1 Tokenization

In this experiment, we implemented a word level tokenization and compared it with a character-level tokenization. Following are the functions for each type of tokenization

```
#-----  
# Character-Level Tokenizer  
# -----  
def char_level_tokenizer(texts, num_words=None):  
    """  
    Create and fit a character-level tokenizer.  
  
    Args:  
        texts (list of str): List of texts.  
        num_words (int or None): Maximum number of tokens to keep.  
  
    Returns:  
        tokenizer: A fitted Tokenizer instance.  
    """  
    tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=num_words, char_level=True, lower=True)  
    tokenizer.fit_on_texts(texts)  
    return tokenizer  
  
#-----  
# Word-Level Tokenizer  
# -----  
def word_level_tokenizer(texts, num_words=None):  
    """  
    Create and fit a character-level tokenizer.  
  
    Args:  
        texts (list of str): List of texts.  
        num_words (int or None): Maximum number of tokens to keep.  
  
    Returns:  
        tokenizer: A fitted Tokenizer instance.  
    """  
    tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=num_words, char_level=False, lower=True)  
    tokenizer.fit_on_texts(texts)  
    return tokenizer
```

Vocabulary Size: The word-level tokenization produced a vocabulary of 80169 unique tokens whereas the character-level tokenization produced a vocabulary of 134 unique tokens.

Each type of tokenization was tested with a base MLP model with one input layer, 2 hidden layers and 1 output layer.

The testing was done with 3 random seeds: 000, 12, and 123 for each type of tokenization for significance testing.

For simplification and less RAM consumption, we performed batch testing by dividing the test set into 10 batches of 2500 samples each.

Table 1 presents the average test accuracy, precision, recall, and loss for all 3 random seeds with both tokenizations.

Table 1: Word-level vs Character-level Tokenization ('RS': Random Seed)

Tokenization	Accuracy	Precision	Recall	loss
Word-level (RS: 123)	0.8648	0.8655	0.8639	0.7104
Character-level (RS: 123)	0.5988	0.6316	0.4746	0.6649
Word-level (RS: 12)	0.8666	0.8693	0.8628	0.7355
Character-level (RS: 12)	0.6099	0.5917	0.7088	0.6591
Word-level (RS: 000)	0.8652	0.8705	0.8580	0.7016
Character-level (RS: 000)	0.6099	0.5917	0.7088	0.6591

Conclusion: The above table shows that the word-level tokenization always outperforms the character-level tokenization, in terms of accuracy. Hence, we chose word-level tokenization for the next set of experiments.

1.2 Hyper Parameter Optimization Using Grid Search

In this experiment, we implemented a Grid Search for finding the best hyperparameters for the Base MLP Model. To do so, we implemented the following hyperparameter grid:

```
# -----  
# Hyperparameter Grid  
# -----  
  
learning_rates = [0.001, 0.0001]  
hidden_layers = [2, 3]  
hidden_sizes = [512,]  
batch_sizes = [128,]  
optimizers_list = ['Adam', 'SGD',]  
activation_functions = ['relu', 'tanh',]  
  
# Create a list of all hyperparameter combinations  
param_grid = list(itertools.product(learning_rates, hidden_layers, hidden_sizes, batch_sizes, optimizers_list, activation_functions))
```

The batch size and hidden size parameters were kept to 1 option whereas the learning_rate, hidden_layers, optimizers and activations were kept to 2 options each.

To apply the activation, the base MLP model was modified to take activation_name as an input and apply the activation using the following function:

```
def apply_activation(self, tensor):  
    """Applies the specified activation function."""  
    if self.activation == 'relu':  
        return tf.nn.relu(tensor)  
    elif self.activation == 'tanh':  
        return tf.nn.tanh(tensor)  
    else:  
        raise ValueError(f"Invalid activation function: {self.activation}")
```

We then trained the models on each of the 16 combinations, $2*2*1*1*2*2 = 16$.

To minimize the compute time, the number of epochs were reduced from 10 to 5.

Each epoch took approximately 2 minutes to run, leading to 160 minutes ~2.67 hours of compute time for each run. We tested each configuration of hyperparameters 3 times while keeping the random seed constant to 12.

Table 2: Validation Accuracies across each parameter combination, Run 1**Best Accuracy highlighted in Yellow**

Learning Rate	Hidden Layers	Hidden Sizes	Batch Sizes	Optimizers	Activation	Accuracy
0.001	2	512	128	Adam	relu	0.8904
0.001	2	512	128	Adam	tanh	0.8842
0.001	2	512	128	SGD	relu	0.5538
0.001	2	512	128	SGD	tanh	0.5664
0.001	3	512	128	Adam	relu	0.8910
0.001	3	512	128	Adam	tanh	0.8852
0.001	3	512	128	SGD	relu	0.5532
0.001	3	512	128	SGD	tanh	0.5178
0.0001	2	512	128	Adam	relu	0.8706
0.0001	2	512	128	Adam	tanh	0.8804

Learning Rate	Hidden Layers	Hidden Sizes	Batch Sizes	Optimizers	Activation	Accuracy
0.0001	2	512	128	SGD	relu	0.5106
0.0001	2	512	128	SGD	tanh	0.5104
0.0001	3	512	128	Adam	relu	0.8634
0.0001	3	512	128	Adam	tanh	0.8800
0.0001	3	512	128	SGD	relu	0.5438
0.0001	3	512	128	SGD	tanh	0.4868

Table 3: validation Accuracies across each parameter combination, Run 2

Learning Rate	Hidden Layers	Hidden Sizes	Batch Sizes	Optimizers	Activation	Accuracy
0.001	2	512	128	Adam	relu	0.8885
0.001	2	512	128	Adam	tanh	0.8823
0.001	2	512	128	SGD	relu	0.5519
0.001	2	512	128	SGD	tanh	0.5645
0.001	3	512	128	Adam	relu	0.8902
0.001	3	512	128	Adam	tanh	0.8843
0.001	3	512	128	SGD	relu	0.5513
0.001	3	512	128	SGD	tanh	0.5169
0.0001	2	512	128	Adam	relu	0.8687
0.0001	2	512	128	Adam	tanh	0.8785
0.0001	2	512	128	SGD	relu	0.5097

0.0001	2	512	128	SGD	tanh	0.5095
0.0001	3	512	128	Adam	relu	0.8615
0.0001	3	512	128	Adam	tanh	0.8781
0.0001	3	512	128	SGD	relu	0.5419
0.0001	3	512	128	SGD	tanh	0.4859

Table 4: validation Accuracies across each parameter combination, Run 3

Learning Rate	Hidden Layers	Hidden Sizes	Batch Sizes	Optimizers	Activation	Accuracy
0.001	2	512	128	Adam	relu	0.8894
0.001	2	512	128	Adam	tanh	0.8832
0.001	2	512	128	SGD	relu	0.5529
0.001	2	512	128	SGD	tanh	0.5654

0.001	3	512	128	Adam	relu	0.8919
0.001	3	512	128	Adam	tanh	0.8861
0.001	3	512	128	SGD	relu	0.5523
0.001	3	512	128	SGD	tanh	0.5187
0.0001	2	512	128	Adam	relu	0.8696
0.0001	2	512	128	Adam	tanh	0.8794
0.0001	2	512	128	SGD	relu	0.5101
0.0001	2	512	128	SGD	tanh	0.5100
0.0001	3	512	128	Adam	relu	0.8624
0.0001	3	512	128	Adam	tanh	0.8790
0.0001	3	512	128	SGD	relu	0.5429
0.0001	3	512	128	SGD	tanh	0.4863

After running the grid search thrice, the best Validation accuracy was approximately 89% mapping the following parameters:

- Learning Rate: 0.001
- Hidden Layers: 3
- Hidden Sizes: 512
- Batch Size: 128
- Optimizer: Adam
- Activation: relu

We then used these best parameters to re-train on the base MLP model on all the training and validation dataset with all 3 random seeds: 000, 12, and 123. We then evaluated the accuracy on the test set using batch testing with 10 batches of 250 samples each. The results were logged in table 5.

Table 5: Best Model performance on the Test Set

Random Seed	Accuracy	Loss	Precision	Recall
12	0.8708	0.4791	0.8736	0.8670
123	0.8716	0.4889	0.8757	0.8662
000	0.8723	0.4837	0.8742	0.8697

This resulted in the following mean accuracy and standard error for the Best MLP Model:

Mean Accuracy = 0.8715

Standard Error = 0.0004

We then Compared This model with the Random MLP with the same hyperparameters. Table 6 shows performance of random MLP with the selected hyperparameters.

Table 6: Random Model performance on the Test Set

Random Seed	Accuracy	Loss	Precision	Recall
12	0.6800	0.6072	0.6645	0.7268
123	0.6872	0.5965	0.6949	0.6675
000	0.6861	0.5988	0.6748	0.7185

This resulted in the following mean accuracy and standard error for the Random MLP Model:

Mean Accuracy = 0.6844

Standard Error = 0.0022

3. Deeper MLP with Feedback Alignment

In this experiment, we implemented a deeper MLP with 5 hidden layers and tested it with the best hyperparameters found in the above experiments. The testing was again performed in batches with 10 batches of 250 samples each.

Following are the results of Deeper MLP with Feedback alignment:

Table 7: Deeper MLP_FA performance on the Test Set

Random Seed	Accuracy	Loss	Precision	Recall
12	0.8722	1.0018	0.8722	0.8524

Discussion:

In this assignment, we learned how Multi-Layer Perceptrons (MLPs) work and how to implement them using algebraic equations. One of the key findings was that word-level tokenization outperformed character-level tokenization in sentiment analysis. This likely happens because words can convey more meaning and context related to sentiment than individual characters. We conducted a grid search to identify the best hyperparameters for our model. The optimal settings included a learning rate of 0.001, three hidden layers, 512 hidden units, a batch size of 128, the Adam optimizer, and ReLU activation. Notably, the choice of optimizer had a significant impact on performance. When we used Stochastic Gradient Descent (SGD), the model's accuracy dropped by about 30%.

Our best MLP model achieved a mean accuracy of 87.15%, with a standard error of 0.0004. In contrast, we also tested a random MLP where only the last layer was trained while keeping the same hyperparameters. This model yielded a lower accuracy of 68.44% with a standard error of 0.002. This comparison highlights the importance of training all layers in an MLP for better performance. However, it also raises questions about efficiency and resource use. The random MLP was quicker to train by 2.6 minutes, which could be beneficial in situations where time is limited. Additionally, we explored a

deeper MLP with Feedback Alignment, which slightly improved our accuracy to 87.22%. This suggests that more complex models can sometimes provide better results, although the improvement was minimal.

Conclusion:

In conclusion, this assignment helped us understand how MLPs function and their application in tasks like sentiment analysis. We discovered that word-level tokenization is more effective than character-level tokenization for capturing sentiment nuances. The hyperparameter tuning process revealed how critical choices, such as the optimizer and the number of layers, can significantly affect model performance. Our findings emphasize the value of training all layers in an MLP to achieve higher accuracy. Furthermore, this project raised important questions about balancing performance with training time and resource efficiency. While simpler models may save time, they may not always deliver the best results.

Overall, this experience has provided valuable insights into building and optimizing MLPs for natural language processing tasks. It also opens up avenues for further exploration in model design and efficiency improvements in future projects.